

**GROUP 10**

# **THE SILENT INVIGILATOR**

**Autonomous Real-Time Exam Malpractice  
Detection System Using Behavioral Analysis**

**PRESENTED BY**

**JUDITH K VIJESH**  
**Reg No: VJC23CS117**

**DAVOOD JAMAL**  
**Reg No: VJC23CS069**

**BENEDICT CHACKO MATHEW**  
**Reg No: VJC23CS062**

**MUHAMMED JAMALUDHEEN U C**  
**Reg No: VJC23CS142**

**GUIDE:**

**Ms.ASHLY GEORGE**  
**Assistant Professor, CSE Dept**

**Date: 20-MAY-2026**

# PRESENTATION OVERVIEW

|   |                                    |   |                                                   |    |                                           |
|---|------------------------------------|---|---------------------------------------------------|----|-------------------------------------------|
| 1 | Objective                          | 5 | Proposed System: User classes and characteristics | 9  | Implementation details of functionalities |
| 2 | Introduction and problem statement | 6 | Proposed System: Architecture Diagram             | 10 | Conclusion                                |
| 3 | Disadvantages of existing system   | 7 | Hardware and software requirements                | 11 | References                                |
| 4 | Proposed System: Main function     | 8 | Gantt chart                                       |    |                                           |

# Objectives

- **BUILD AN AI-POWERED INVIGILATION ASSISTANT FOR LIVE EXAMINATION MONITORING**
- **DETECT SUSPICIOUS BEHAVIOR AND PROHIBITED OBJECTS IN REAL TIME**
- **REDUCE HUMAN ERROR, FATIGUE, AND INCONSISTENCY IN MANUAL SUPERVISION**
- **GENERATE STRUCTURED ALERTS AND EVIDENCE-BACKED SESSION LOGS**
- **PRESERVE PRIVACY THROUGH LOCAL-FIRST PROCESSING ARCHITECTURE**



# INTRODUCTION AND PROBLEM STATEMENTS

- Traditional invigilation is manual, subjective, and difficult to scale consistently
- Short-duration suspicious actions are often missed in crowded exam halls
- Institutions require a real-time, objective, and auditable monitoring pipeline
- The system must detect behavior anomalies and object-based malpractice cues
- The Silent Invigilator addresses this with integrated CV + DL intelligence

# Disadvantages of existing system

- **High dependence on human attention and endurance during long sessions**
- **Inconsistent judgment quality across invigilators and exam environments**
- **Limited real-time evidence capture for post-exam academic review**
- **Low scalability for multi-candidate, multi-session monitoring operations**
- **Manual logs are delayed, fragmented, and difficult to audit reliably**

# **Proposed System- Main Function**

- **Real-time video capture, frame processing, and live stream delivery**
  - **Head pose and gaze analysis using facial landmarks and iris geometry**
  - **YOLOv8-based prohibited object detection (phone and suspicious materials)**
  - **Multi-student tracking with persistent IDs and temporal risk scoring**
  - **Role-based dashboards with alerting, acknowledgement, and report APIs**
- 



# Proposed System

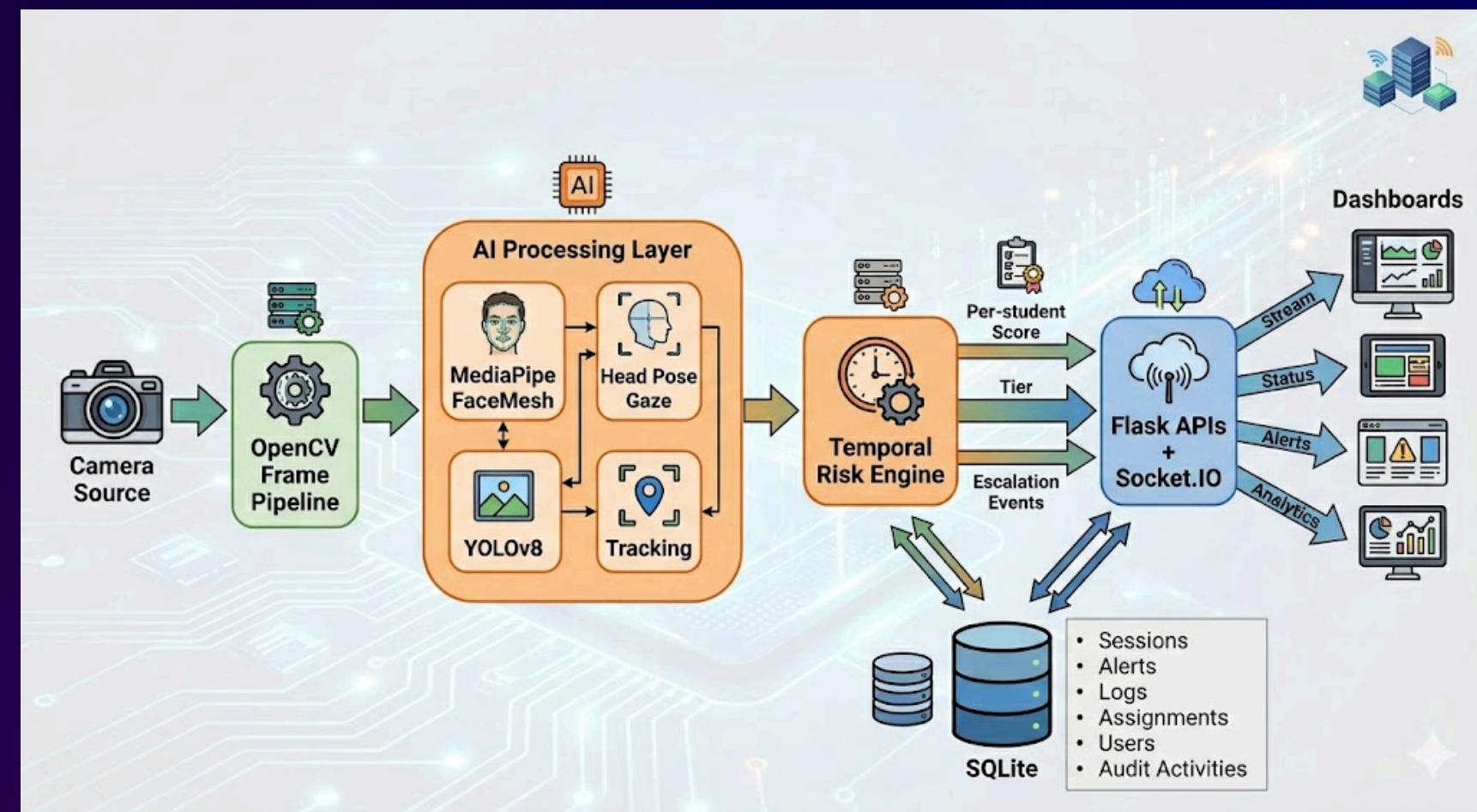
User classes and characteristics

- **Exam Invigilator: Monitors live sessions and responds to detected incidents**
- **System Administrator / Academic Review Staff:**
  1. **Manages users, sessions, assignments, and audit visibility**
  2. **Evaluates reports, timelines, and suspicious patterns**
- **Each role receives purpose-specific access and workflow-aligned interfaces**
- **Decision authority remains human; AI functions as precision augmentation**

# Proposed System Architecture Diagram

**Camera Source -> OpenCV Frame Pipeline -> AI Processing Layer**

- AI Layer: MediaPipe FaceMesh + Head Pose + Gaze + YOLOv8 + Tracking
- Temporal Risk Engine generates per-student score, tier, and escalation events
- Flask APIs + Socket.IO deliver stream, status, alerts, and analytics to dashboards
- SQLite persists sessions, alerts, logs, assignments, users, and audit activities



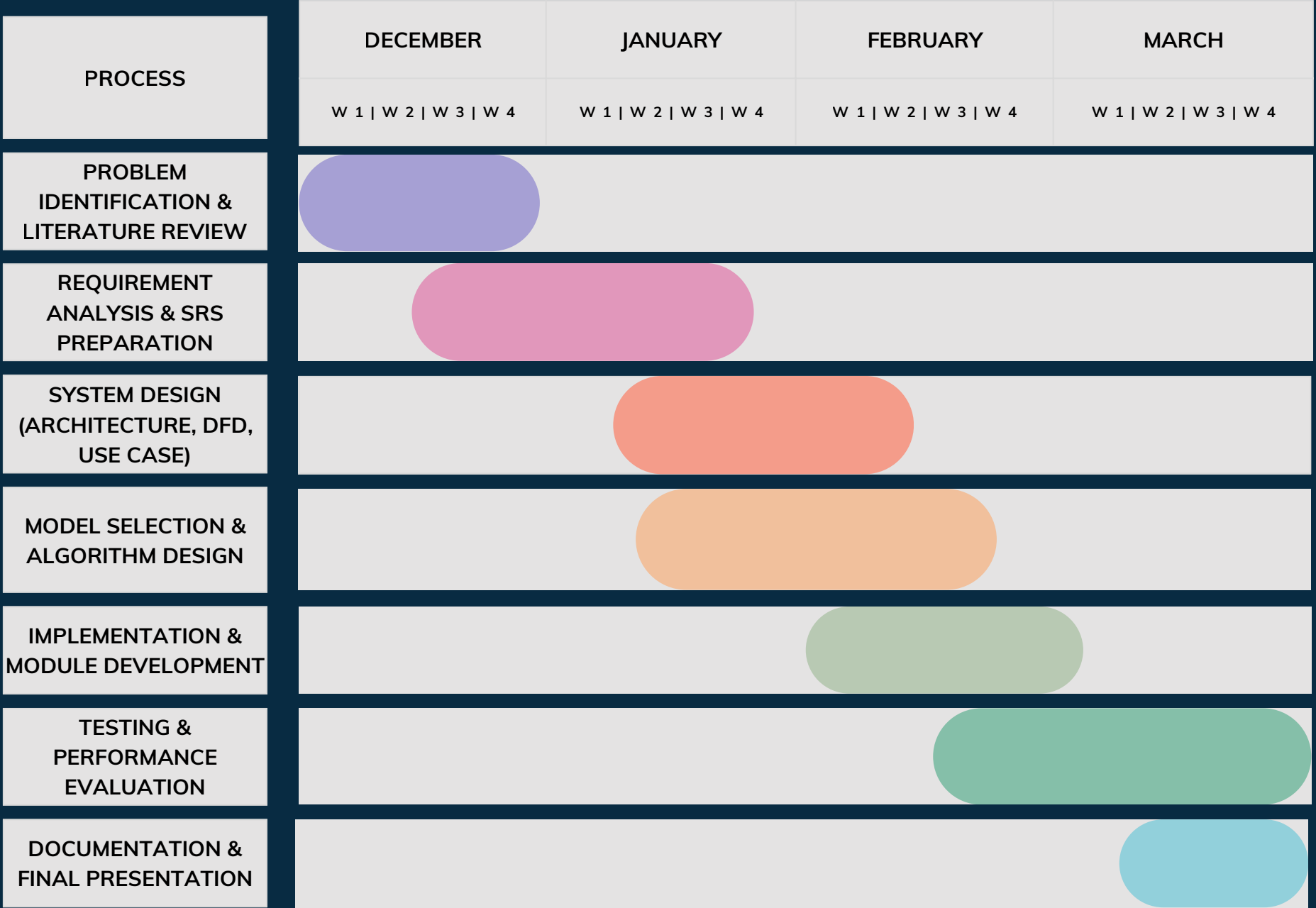


# Hardware and Software Requirements

- Hardware: Webcam/IP camera, 8 GB+ RAM, multi-core CPU, optional CUDA GPU.
- Operating System: Windows 10+ (current project environment).
- Core Stack: Python 3.10+, OpenCV, MediaPipe, Ultralytics YOLOv8, SAHI.
- Platform Stack: Flask, Flask-SocketIO, SQLite, JWT, bcrypt, HTML/CSS/JS.
- Client Extension: Flutter mobile monitoring interface with API integration

# Gantt Chart

GANTT CHART  
PROCESS TIMELINE



# Implementation Details

## Algorithms and Equations

### FRAME PROCESSING PIPELINE

Real-time multi-stage pipeline for robust monitoring:

- Video Capture: OpenCV frame acquisition
- Face Analysis: MediaPipe FaceMesh → facial landmarks extraction
- Object Detection: YOLOv8 → detect suspicious objects
- Tracking: ByteTrack (fallback: IoU-based tracking)
- Behavior Analysis: Head pose + gaze estimation
- Risk Evaluation: Multi-factor scoring system
- Alert Engine: Temporal buffering + cooldown + escalation policy

### Head Pose Equation

Projects 3D facial points to 2D image plane:

$$x_i = K[R|t]X_i \quad x_i = K[R \mid t]X_i \quad x_i = K[R|t]X_i$$

PnP Solution:

$$[rvec, tvec] = solvePnP(X, x, K, D)$$

$$R = Rodrigues(rvec)$$

$$angles = RQDecomp3x3(R)$$



# Implementation Details

## Algorithms and Equations

### Gaze Estimation

Determines eye movement direction using normalized iris position:

$$ratio = ||p_{iris} - p_{outer}|| / (||p_{inner} - p_{outer}|| + \varepsilon)$$

Classification Logic:

- $ratio < T_1 \rightarrow \text{Left}$
- $T_1 \leq ratio \leq T_2 \rightarrow \text{Center}$
- $ratio > T_2 \rightarrow \text{Right}$

### Risk Scoring Model

Weighted combination of behavioral indicators:

$$S = clip(100(0.40 \cdot object + 0.22 \cdot gazedev + 0.16 \cdot headdev + 0.16 \cdot headdev + 0.14 \cdot lookdown + 0.08 \cdot sustained), 0, 100)$$

### Tracking & Alert Logic

Tracking: ByteTrack ensures stable student identity across frames

Fallback: IoU-based tracking when confidence drops

Temporal Buffers: Reduce false positives

Cooldown Mechanism: Prevent alert spamming

Escalation Policy: Repeated violations  $\rightarrow$  higher alert level

# Implementation and Details

## SQLite Tables in Use

- **Access and Identity:**  
roles, users, refresh\_tokens, access\_token\_denylist Supports RBAC, account lifecycle, JWT refresh and revocation
- **Session Control:**  
exam\_sessions, session\_assignments, Supports session creation, invigilator assignment, start/complete lifecycle, access scoping
- **Monitoring and Evidence:**  
alerts (primary session-scoped evidence), logs (legacy/summary operational events used in dashboards)
- **Governance and Audit:**  
Activity\_logs for user-action traceability (login, session operations, acknowledgements, stream access)



# Implementation Details

## Functionality vs Components

|   |                                                                                                       |
|---|-------------------------------------------------------------------------------------------------------|
| 1 | Behavior Analysis: MediaPipe + OpenCV + NumPy (head pose, gaze, face state)                           |
| 2 | Object Detection: Ultralytics YOLOv8 (+ SAHI slicing in code) for phone/material detection            |
| 3 | Real-Time Platform: Flask + Flask-SocketIO + MJPEG stream + status/report APIs                        |
| 4 | Security Layer: bcrypt password hashing + JWT access/refresh + role-guarded routes                    |
| 5 | Hardware/Software per Function: webcam/IP camera, Windows system, Python 3.10+, SQLite-backed backend |



# Conclusion

- The Silent Invigilator delivers a complete AI-assisted exam monitoring workflow
- Implemented pipeline supports detection, tracking, risk scoring, and alert escalation
- Operational dashboards and secure APIs provide real-time and post-exam intelligence
- Architecture is modular, scalable, and aligned to institutional integrity needs
- Final outcome: faster, fairer, and evidence-driven invigilation support

## **REFERENCES & BIBLIOGRAPHY**

- [1] IEEE Computer Society, IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications, IEEE, 1998**
- [2] A. Saravanan et al., “Deep Learning-Based Smart Invigilation System for Enhanced Exam Integrity,” International Journal of Computer Trends and Technology, 2024**
- [3] M. Cote et al., “Automated Detection of Suspicious Behavior During Online Exams Using Artificial Intelligence and Computer Vision,” CEUR Workshop Proceedings, Vol-4036, 2025**
- [4] S. Khatoon et al., “Smart Proctor: A Computer Vision Approach to Online Exam Monitoring,” IRJET, Vol. 12, Issue 11, 2025**
- [5] Ultralytics, YOLOv8 Technical Documentation, 2024**
- [6] Google, MediaPipe Face Mesh Documentation, 2024**